# JavaScript from Basics — For a Python Developer

*Learning JS by mapping it directly onto what you already know in Python*

Prepared for Mohanraj R | Format: Theory → Syntax → Code → Explanation, with Python comparisons throughout

> **How to read this**
> Every concept is explained the way it would click for you fastest — by comparing directly to the Python you already know. Where the two languages think differently (not just different syntax, but a different mental model), that gets called out specifically.

## 1. The big-picture differences first

| Python | JavaScript |
|---|---|
| Runs via an interpreter you install (CPython) | Runs inside a browser, or via Node.js on a server/your machine |
| Indentation defines blocks | Curly braces `{ }` define blocks; indentation is just style, not required |
| Statements don't need a terminator | Statements typically end with a semicolon `;` (optional, but always use it — avoids subtle bugs) |
| Dynamically typed, single "type" system | Dynamically typed in plain JS, but has quirky type coercion (see Section 8) |
| One way to declare variables | Three: `let`, `const`, `var` (avoid `var`) |
| Lists, dicts, tuples, sets | Arrays and objects cover most of that ground |
| `None` | `null` AND `undefined` — two different "nothing" values |
| `def` for functions | `function` keyword, or arrow functions `=>` |
| Single-threaded, blocking by default | Single-threaded but non-blocking — async is baked into the language (Promises, async/await) |

## 2. Variables

**SYNTAX**

```
# Python
name = "Mohanraj"
age = 25
age = 26          # fine, just reassign
```

```
// JavaScript
let name = "Mohanraj";
let age = 25;
age = 26;                 // fine, let allows reassignment

const city = "Chennai";   // cannot be reassigned
// city = "Delhi";        // ✗ TypeError
```

**EXPLANATION**

Python has one way to make a variable. JavaScript makes you choose intent up front: use `const` by default for anything you won't reassign (most values), and `let` only when you know the value will change (loop counters, running totals). You'll see `var` in older code — avoid it; it has confusing scoping rules that `let` / `const` fixed.

> **Important distinction**
> `const` means the *variable binding* can't be reassigned — it does NOT mean the value is frozen. You can still mutate an array or object stored in a `const`:

```
const scores = [90, 85];
scores.push(70);      // ✓ totally fine — mutating contents, not reassigning
// scores = [1,2,3];  // ✗ error — this IS reassignment
```

## 3. Data types

**THEORY**

JavaScript's primitive types: `string`, `number` (no separate int/float — just one number type), `boolean`, `null`, `undefined`, and `bigint` / `symbol` (rare). Everything else — arrays, objects, functions — is an "object" under the hood.

| Python | JavaScript | Note |
|---|---|---|
| `int, float` | `number` | JS has just one numeric type for both |
| `str` | `string` | Same idea, single or double quotes both work in JS |
| `bool (True/False)` | `boolean (true/false)` | lowercase in JS |
| `None` | `null / undefined` | see Section 9 — this trips everyone up at first |
| `list` | `Array` | ordered, mutable, similar methods |
| `dict` | `Object (or Map)` | see Section 6 |
| `tuple` | `(no direct equivalent — use array or Object.freeze)` | |
| `set` | `Set` | similar usage |

**CODE — checking type**

```
// Python: type(x)
typeof "hello";     // "string"
typeof 42;           // "number"
typeof true;          // "boolean"
typeof undefined;      // "undefined"
typeof null;            // "object"  ← famous JS quirk, a known bug kept for compatibility
typeof [1,2,3];           // "object" ← arrays are objects; use Array.isArray() instead
Array.isArray([1,2,3]);    // true
```

## 4. Strings

**SYNTAX**

**PYTHON**

```
name = "Mo"
age = 25
msg = f"{name} is {age}"
print(msg.upper())
print(len(name))
print(name[0])
print(name + " R")
```

**JAVASCRIPT**

```
const name = "Mo";
const age = 25;
const msg = `${name} is ${age}`;
console.log(msg.toUpperCase());
console.log(name.length);
console.log(name[0]);
console.log(name + " R");
```

**EXPLANATION**

Python's f-strings map directly to JavaScript **template literals** — backticks instead of quotes, and `${...}` instead of `{...}`. `.length` is a property (no parentheses) where Python uses the `len()` function — this trips people up constantly at first.

## 5. Arrays (Python's lists)

**SYNTAX**

**PYTHON**

```
nums = [1, 2, 3]
nums.append(4)
nums.pop()
nums[0]
len(nums)
nums.remove(2)
"a" in ["a","b"]
sorted(nums)
```

**JAVASCRIPT**

```
let nums = [1, 2, 3];
nums.push(4);
nums.pop();
nums[0];
nums.length;
nums.splice(nums.indexOf(2), 1);
["a","b"].includes("a");
nums.sort();
```

**EXPLANATION**

`splice` is the odd one — it's JS's swiss-army-knife for inserting/removing at any position: `arr.splice(startIndex, deleteCount, ...itemsToInsert)`. Also note: JS's default `.sort()` sorts as strings unless you pass a comparator — `[10, 2, 1].sort()` gives `[1, 10, 2]`, not what you'd expect. Always pass one for numbers:

```
nums.sort((a, b) => a - b);   // ascending, numeric
```

### The methods you'll use constantly: map / filter / reduce

**CODE**

**PYTHON**

```
nums = [1, 2, 3, 4]
doubled = [n * 2 for n in nums]
evens = [n for n in nums if n % 2 == 0]
total = sum(nums)
```

**JAVASCRIPT**

```
const nums = [1, 2, 3, 4];
const doubled = nums.map(n => n * 2);
const evens = nums.filter(n => n % 2 === 0);
const total = nums.reduce((sum, n) => sum + n, 0);
```

JS has no list-comprehension syntax — `.map()`, `.filter()`, and `.reduce()` together cover what comprehensions do in Python. These three methods are used constantly in real React/Node code, so get comfortable with them early.

## 6. Objects (Python's dictionaries)

**SYNTAX**

**PYTHON**

```
student = {
    "name": "Mo",
    "age": 25
}
print(student["name"])
student["email"] = "mo@x.com"
for key, value in student.items():
    print(key, value)
"name" in student
```

**JAVASCRIPT**

```
const student = {
  name: "Mo",
  age: 25
};
console.log(student.name);       // or student["name"]
student.email = "mo@x.com";
for (const key in student) {
  console.log(key, student[key]);
}
"name" in student;
```

**EXPLANATION**

The big difference: JS objects let you access properties with **dot notation** (`student.name`), not just brackets. Dot notation is the standard style — use brackets only when the key is dynamic (stored in a variable) or not a valid identifier.

```
const key = "age";
student[key];     // dynamic access — dot notation can't do this: student.key would look for a literal "key" property
```

## 7. Conditionals

**SYNTAX**

**PYTHON**

```
age = 20
if age >= 18:
    print("adult")
elif age >= 13:
    print("teen")
else:
    print("child")
```

**JAVASCRIPT**

```
let age = 20;
if (age >= 18) {
  console.log("adult");
} else if (age >= 13) {
  console.log("teen");
} else {
  console.log("child");
}
```

Conditions need parentheses, blocks need braces — that's the whole difference in logic. `elif` becomes `else if`.

## 8. Equality: a JavaScript trap Python doesn't have

**THEORY**

> **Rule: always use === and !==, never == or !=**
>
> `==` in JavaScript does "loose equality" — it tries to convert types before comparing, producing surprising results. `===` ("strict equality") compares value AND type, which is what Python's `==` already does for you by default.

```
0 == false;        // true   ← loose equality coerces types
"" == false;         // true
null == undefined;    // true
0 === false;            // false  ← strict equality, no coercion — use this
"5" === 5;              // false  ← different types, correctly not equal
```

## 9. null vs undefined

**THEORY**

Python has one "nothing" value: `None`. JavaScript has two, and the difference matters:

| Value | Meaning |
|---|---|
| `undefined` | A variable was declared but never given a value — JS's automatic default. Also what a missing object property or missing function argument resolves to. |
| `null` | Explicitly set by you (or an API) to mean "intentionally empty." You have to assign this yourself — JS never assigns it automatically. |

```
let x;
console.log(x);           // undefined — never assigned

let user = null;          // you're explicitly saying "no user yet"

// safely reading nested properties that might not exist:
const city = user?.address?.city;    // optional chaining — undefined instead of a crash
const name = user?.name ?? "Guest";  // nullish coalescing — fallback only for null/undefined
```

`??` is close to Python's `value or default`, but safer — it only falls back on `null` / `undefined`, not on falsy values like `0` or `""`, which `or` would incorrectly replace.

## 10. Loops

**SYNTAX**

**PYTHON**

```
for i in range(5):
    print(i)

fruits = ["apple", "mango"]
for fruit in fruits:
    print(fruit)

for i, fruit in enumerate(fruits):
    print(i, fruit)

i = 0
while i < 5:
    print(i)
    i += 1
```

**JAVASCRIPT**

```
for (let i = 0; i < 5; i++) {
  console.log(i);
}

const fruits = ["apple", "mango"];
for (const fruit of fruits) {
  console.log(fruit);
}

fruits.forEach((fruit, i) => {
  console.log(i, fruit);
});

let i = 0;
while (i < 5) {
  console.log(i);
  i++;
}
```

**EXPLANATION**

Two different "for" loops matter: `for...of` iterates over **values** (like Python's `for x in list`) and `for...in` iterates over **keys/indices** — easy to mix up and a common bug source. For arrays, prefer `for...of` or `.forEach()`; save `for...in` for objects.

## 11. Functions

**SYNTAX**

**PYTHON**

```python
def add(a, b):
    return a + b

def greet(name, greeting="Hello"):
    return f"{greeting}, {name}"

square = lambda x: x * x
```

**JAVASCRIPT**

```javascript
function add(a, b) {
  return a + b;
}

function greet(name, greeting = "Hello") {
  return `${greeting}, ${name}`;
}

const square = (x) => x * x;   // arrow function ~ lambda
```

**EXPLANATION**

Arrow functions ( `=>` ) are JS's version of Python's `lambda`, but far more commonly used — they're the default style for short functions, callbacks, and array methods, not just one-liners.

```javascript
// common real-world pattern — arrow functions as callbacks
const doubled = [1, 2, 3].map((n) => n * 2);
button.addEventListener("click", () => {
  console.log("clicked");
});
```

> **One real behavioral difference: arrow functions and "this"**
> Regular `function` declarations get their own `this` context; arrow functions inherit `this` from where they're written. This has no Python parallel — you'll run into it once you start writing classes and event handlers, and it's a very common source of bugs for beginners. For now: prefer arrow functions for callbacks, regular functions for object/class methods.

## 12. Destructuring & spread — JS's answer to unpacking

**SYNTAX**

**PYTHON**

```python
a, b = 1, 2

first, *rest = [1, 2, 3, 4]

def total(*args):
    return sum(args)

person = {"name": "Mo", "age": 25}
name = person["name"]
```

**JAVASCRIPT**

```javascript
let [a, b] = [1, 2];

const [first, ...rest] = [1, 2, 3, 4];

function total(...args) {
  return args.reduce((s, n) => s + n, 0);
}

const person = { name: "Mo", age: 25 };
const { name } = person;   // object destructuring — very common
```

**EXPLANATION**

`...` is JS's version of Python's `*args` — called "rest" when collecting, "spread" when expanding:

```javascript
const nums = [1, 2, 3];
const combined = [...nums, 4, 5];      // spread — like Python's [*nums, 4, 5]
const obj2 = { ...person, age: 26 };    // spread into a new object — common for immutable updates
function log(...args) { console.log(args); }  // rest — collects extra args into an array
```

Object destructuring ( `const { name } = person` ) is used everywhere in React — that's exactly what's happening when a component receives `{ title, onClick }` as props, which you saw in the TypeScript guide.

## 13. Classes

**SYNTAX**

**PYTHON**

```python
class Animal:
    def __init__(self, name):
        self.name = name

    def speak(self):
        print(f"{self.name} makes a sound")

class Dog(Animal):
    def speak(self):
        print(f"{self.name} barks")

d = Dog("Rex")
d.speak()
```

**JAVASCRIPT**

```javascript
class Animal {
  constructor(name) {
    this.name = name;
  }
  speak() {
    console.log(`${this.name} makes a sound`);
  }
}

class Dog extends Animal {
  speak() {
    console.log(`${this.name} barks`);
  }
}

const d = new Dog("Rex");
d.speak();
```

`__init__` becomes `constructor`, `self` becomes `this`, and you must use `new` when creating an instance — Python never requires that keyword.

## 14. Modules — import/export

**SYNTAX**

**PYTHON**

```python
# mathutils.py
def add(a, b):
    return a + b

# main.py
from mathutils import add
```

**JAVASCRIPT**

```javascript
// mathUtils.js
export function add(a, b) {
  return a + b;
}
export default function multiply(a, b) { return a * b; }

// main.js
import multiply, { add } from "./mathUtils.js";
```

A file can have exactly one `export default` (imported without curly braces) and any number of named exports (imported with curly braces, matching the exact name).

## 15. Async code — the biggest conceptual jump from Python

**THEORY**

Python code normally runs top to bottom, blocking on things like network requests unless you specifically use `asyncio`. JavaScript is non-blocking by default — anything that takes time (API calls, timers, file reads in Node) doesn't freeze the rest of the program. This is core to the language, not an optional add-on.

**SYNTAX**

**PYTHON (asyncio)**

```python
import asyncio

async def get_user():
    data = await fetch_from_api()
    return data

asyncio.run(get_user())
```

**JAVASCRIPT**

```javascript
async function getUser() {
  const response = await fetch("/api/user");
  const data = await response.json();
  return data;
}

getUser();
```

**EXPLANATION**

The syntax is almost identical to Python's asyncio — `async` before the function, `await` before anything that takes time. The difference is that in JS, you'll use this constantly and by default (every API call, every database query in Node), not as a special opt-in mode.

```javascript
// what await is actually unwrapping — a Promise
function getUserPromiseStyle() {
  return fetch("/api/user")
    .then((response) => response.json())
    .then((data) => console.log(data))
    .catch((error) => console.error(error));
}
// async/await is cleaner syntax over the same Promise mechanism above
```

## 16. Quick reference — syntax lookup table

| Python | JavaScript |
|---|---|
| `print(x)` | `console.log(x)` |
| `len(x)` | `x.length` |
| `f"{x}"` | `` `${x}` `` |
| `None` | `null / undefined` |
| `True / False` | `true / false` |
| `and / or / not` | `&& / \|\| / !` |
| `elif` | `else if` |
| `def` | `function / =>` |
| `self` | `this` |
| `__init__` | `constructor` |
| `list.append(x)` | `arr.push(x)` |
| `[x for x in y]` | `y.map(x => x)` |
| `dict.items()` | `Object.entries(obj)` |
| `*args` | `...args (rest)` |
| `import x from y` | `import { x } from "y"` |
| `a == b` | `a === b (never ==)` |
| `try / except` | `try / catch` |
| `raise Error(...)` | `throw new Error(...)` |

---

**Next step:** Once this is comfortable, JavaScript + TypeScript will click fast — TypeScript is just this language with types layered on top, which is exactly what your other guide already covers. Practice by rewriting 3–4 small Python scripts you've already written (a calculator, a simple data filter) directly in JavaScript, side by side.